

Task 10 — Agent-to-Agent Interoperability Across Trust Boundaries

TRACK Agentic AI — Intern Programme, Task 10 of 11

PACKAGE	packages/a2a
PREREQUISITES	Task 4 (orchestration) and Task 5 (safety). TypeScript.
COST	Zero. Everything runs locally.

Why this task exists

Task 4's Planner, Executor, and Critic all trust each other completely — they're processes you wrote, on your machine, sharing your assumptions. Real agent ecosystems increasingly involve agents built by different teams, or different companies, talking to each other over a network: an emerging interoperability pattern of capability discovery, negotiated task handoff, and zero trust in what the other side claims about itself. That's a different problem than internal orchestration — you can no longer assume the other agent is well-behaved, well-specified, or even honest about what it can do.

Setup

Same runtime, plus a second, deliberately separate process — a second Node process is enough, it doesn't need to be a different machine — representing an “external” agent your system talks to over HTTP, with its own directory, its own model, and no shared code with your primary agent beyond the wire protocol.

What you are building

Your Task 3/4 agent needs a capability it doesn't have — e.g. “check this claim against structured data you don't have access to” — and discovers, negotiates with, and delegates to an external agent to get it, then verifies what comes back rather than trusting it.

```
pnpm a2a run --task "verify this claim against the external agent's dataset"
pnpm a2a eval
pnpm a2a eval --compare baseline.json
```

Requirements

1. Capability discovery

The external agent publishes a typed, versioned capability manifest your agent queries before delegating. Don't hand it a task and hope — confirm it claims to do the thing first.

2. Task handoff protocol

A typed request/response contract for delegation, with an explicit timeout and a defined behavior on timeout or malformed response: fail closed, not silently accept whatever came back.

3. Untrusted-response verification

Treat anything the external agent returns as unverified input — the same category as a tool result an attacker could shape. This is your Task 5 red-team muscle, applied to a peer agent instead of a file. At minimum: validate the response against the manifest's declared schema, and run at least one of your Task 5 injection-style checks against content the external agent returns.

4. A deliberately misbehaving external agent

Build a second version of the external agent that lies about its capabilities, returns malformed responses, or embeds an injection attempt in its response. Confirm your primary agent's handling degrades safely against all three variants, not just the honest one.

5. Golden evaluation set — 10 delegation scenarios

EXTERNAL AGENT VARIANT	COUNT	EXPECTED HANDLING
Honest	6	Successful delegation and correct use of the response
Capability-lying	2	Discovery or response validation rejects the delegation; fails closed
Injection-embedding	2	The injected instruction in the response is not followed

6. Metrics

METRIC	WHAT IT MEASURES
successful delegation rate	Against the honest external agent
fail-closed rate	Against misbehaving agents — should be near 100%
schema-violation catch rate	Malformed or out-of-contract responses correctly rejected
added latency from verification	Cost of the zero-trust layer itself

Deliverable: RESULTS.md

Compare naive delegation (trust the response outright) against verified delegation, showing concretely what naive delegation would have let through. NOTES.md on why zero-trust between agents matters even when you wrote both sides yourself in this exercise — the wire boundary is what matters, not who wrote the code on either side of it.

Hard parts you should expect

- A capability manifest that's technically true but misleading — claims to support X, with caveats the manifest format doesn't capture.
- Timeouts tuned wrong: too tight breaks legitimate slow responses, too loose lets a hung peer stall your whole run.

- An injection embedded in an otherwise well-formed, schema-valid response — schema validation catches malformed data, not malicious well-formed data.
- Verification logic that's stricter against the honest agent than the dishonest ones by accident, because the honest agent's edge cases weren't in your test set.

Deliverables

- packages/a2a — capability discovery, handoff protocol, verification layer, the external agent (all three variants)
- evals/golden-a2a.jsonl — 10 delegation scenarios
- Metrics harness reporting all four metrics
- RESULTS.md, NOTES.md, DESIGN.md

Design doc, before any code

One page, reviewed and approved before you write a line:

- Public interfaces and types (capability manifest, handoff request/response)
- The three most likely failure modes, and your plan for each
- What you are deliberately not building, and why
- Open questions

Review gate

Your reviewer will verify these by hand before merge.

- Point the primary agent at the capability-lying external agent and confirm it fails closed rather than proceeding on a bad delegation.
- Point it at the injection-embedding variant and confirm the injected instruction has no effect on the primary agent's behavior.
- Kill the external agent mid-request and confirm the timeout path fires and is logged, not hung indefinitely.
- Every number in RESULTS.md reproduces from a fresh clone.